

Week 2 Day 2: Dockerfile과 이미지 빌드

Overview

Day 2는 Day 1에서 실행한 container를 직접 만들 수 있도록 Dockerfile과 image build를 다룬다. 학생은 표준 실습 앱의 소스 구조를 확인하고, Dockerfile로 image를 만든 뒤, container 실행과 HTTP evidence로 결과를 검증한다.

오늘의 핵심 질문은 다음과 같다.

내 소스 파일과 실행 조건을 Dockerfile에 어떻게 기록해야 다른 사람이 같은 image를 만들고 실행할 수 있는가?

Day 2는 단순히 docker build 명령을 따라 치는 날이 아니다. image는 read-only layer의 누적이고, container는 그 image 위에 writable layer를 붙여 실행된다. build context는 Docker daemon에게 전달되는 파일 범위이며, .dockerignore는 그 범위를 줄이는 재현성/보안 장치다. 또한 bind mount와 named volume은 container filesystem 바깥에 데이터를 두는 방법이므로, image build와 container data persistence를 구분해야 한다.

Learning Goals

- image layer, cache, tag, digest의 의미를 설명한다.
- FROM, WORKDIR, COPY, RUN, CMD, EXPOSE의 역할을 구분한다.
- 표준 실습 앱의 build context와 .dockerignore 범위를 확인한다.
- docker build로 image를 만들고 docker run으로 실행한다.
- curl, docker logs, docker exec, docker history, docker inspect로 build/run 결과를 검증한다.
- image tag와 Docker Hub 사용 시 보안/공개 범위/latest 위험을 설명한다.
- README에 build/run/check/cleanup/troubleshoot 명령을 기록한다.

Lesson Index

- 1교시: 이미지와 레이어의 이해 - base image, layer, cache, tag, digest
- 2교시: Dockerfile 기본 문법 - FROM, WORKDIR, COPY, RUN, CMD, EXPOSE
- 3교시: 표준 실습 앱 소스코드 확인 - file tree, build context, .dockerignore
- 4교시: 표준 실습 앱 이미지 만들기 - docker build, docker run
- 5교시: 이미지 빌드 문제 해결 - context, cache, path, permission
- 6교시: 컨테이너 실행 검증 - port, logs, docker exec, 내부 파일 확인
- 7교시: 디스크 연동 1 - bind mount로 host 파일을 container에서 서빙
- 8교시: 디스크 연동 2 - named volume 데이터 보존과 README evidence

Practice Files And Assets

자료	용도
labs/static-site/index.html	표준 실습 앱 HTML
labs/static-site/styles.css	표준 실습 앱 CSS
labs/static-site/Dockerfile	image build 실습 파일
labs/static-site/.dockerignore	build context 제외 규칙
labs/static-site/README.md	실습 앱 build/run/check/cleanup 예시
hands-on-lab.md	Day 2 전체 긴 실습 절차와 failure drill
assets/lesson-01-image-layers-cache-tag-digest.png	image layer/cache/tag/digest 설명
assets/lesson-02-dockerfile-instruction-map.png	Dockerfile instruction map
assets/lesson-03-source-tree-build-context.png	source tree와 build context
assets/lesson-04-build-run-pipeline.png	build/run/HTTP check pipeline
assets/lesson-05-build-troubleshooting.png	build failure triage
assets/lesson-06-container-verification.png	container 실행 검증
assets/lesson-07-disk-bind-mount-nginx.png	bind mount로 host disk와 nginx container 연결
assets/lesson-08-named-volume-readme-evidence.png	named volume 데이터 보존과 README evidence
labs/disk-mount/html/index.html	bind mount 실습 HTML
labs/disk-mount/data/note.txt	host disk note 예시

Linux Preflight Test Evidence

Day 2 build/run 명령은 Linux 환경에서 사전 테스트했다.

항목	결과
Test OS	Ubuntu 24.04.3 LTS, Linux 6.6.87.2-microsoft-standard-WSL2, linux/amd64
Docker version	Client 29.0.2, Server 29.3.1
Build command	<code>docker build -t paperclip-static-site:day2 .</code>
Build result	image sha256:2769096e..., build context 2.42kB, 성공
Run command	<code>docker run -d --name paperclip-day2-static -p 18082:80 paperclip-static-site:day2</code>
HTTP check	<code>curl -I http://localhost:18082 -> HTTP/1.1 200 OK, Server: nginx/1.27.5</code>
Source check	<code>curl -s</code> 에서 Dockerfile로 만든 표준 실습 앱 확인
Internal file check	<code>docker exec ... ls -l /usr/share/nginx/html</code> 에서 index.html, styles.css 확인
Image tag check	paperclip-static-site:day2와 paperclip-static-site:day2-verified가 같은 image ID
Layer/cache metrics	cache build 1.54 sec, 전체 image 48.2MB, COPY layer 2.34kB, 변경 layer 비율 약 0.0049%
Bind mount check	paperclip-day2-disk가 host path를 /usr/share/nginx/html:ro로 mount, v1 -> v2 응답 변경 확인
Named volume check	paperclip-day2-data에 volume-note-v1 write 후 새 container에서 read 성공
Cleanup	<code>docker stop / docker rm</code> 성공, recheck 시 container 없음

Required Evidence

Evidence	제출 기준
Dockerfile	instruction별 역할을 설명할 수 있는 파일
Build output	build context, COPY step, image tag 기록
Run output	container ID, <code>docker ps</code> PORTS 기록
HTTP evidence	HTTP/1.1 200 OK 또는 browser screenshot filename
Internal file evidence	<code>docker exec</code> 로 내부 파일 확인
Layer/cache evidence	cache build 시간, CACHED step, 전체 image 크기, COPY layer 크기 기록
Tag evidence	local image tag 2개와 image ID 비교
Disk evidence	bind mount source/destination, named volume persistence 확인
README update	build/run/check/logs/cleanup/troubleshoot 포함

Extended Hands-on Scope

Day 2의 상세 실습은 [hands-on-lab.md](#)를 기준으로 진행한다. lesson 본문은 교시별 설명과 핵심 활동을 제공하고, lab guide는 전체 실행 명령, 기대 출력, failure drill, cleanup audit을 제공한다.

Phase	실습 범위	연결 교시
A	실습 앱 구조와 build context 확인	3교시
B	Dockerfile instruction 분석	2교시
C	image build와 cache 재빌드	4~5교시
D	container run, HTTP, logs, exec 검증	4~6교시
E	COPY/path/port/cache failure drill	5교시
F	bind mount disk 연동 v1/v2	7교시
G	named volume persistence	8교시
H	README evidence와 cleanup audit	8교시

Cost And Security Notes

- Day 2는 local Docker와 public base image만 사용한다. cloud resource를 만들지 않는다.
- Docker Hub push는 선택 시연 또는 설명 중심이다. 개인 계정 credential, token, MFA code를 기록하지 않는다.
- .dockerignore는 build context에 secret, log, screenshots, dependency cache가 들어가지 않도록 관리하는 보안/효율 장치다.
- bind mount는 host path를 직접 노출하므로 source path와 read/write mode를 확인한다.
- named volume은 container 삭제 후에도 데이터가 남을 수 있으므로 `docker volume rm` 전 데이터를 확인한다.

Academic And Operational Depth Map

개념	학술/시스템 관점	Docker 실습 연결	운영 질문
Image layer	union filesystem과 copy-on-write 계열 사고	docker history, COPY layer	어떤 변경이 image size와 rebuild 시간을 늘렸는가
Build context	build 입력 집합과 재현 가능한 source boundary	docker build ., .dockerignore	secret이나 불필요한 파일이 daemon에 전달되는가
Cache	instruction 결과 재사용	build output, --no-cache	cache 때문에 오래된 결과를 보고 있지 않은가
Tag/digest	이름표와 content identity 구분	paperclip-static-site:day2, image ID	같은 tag가 같은 내용을 뜻한다고 가정해도 되는가
Container writable layer	image와 실행 중 변경 상태 분리	container 생성/삭제	container 삭제 시 어떤 변경이 사라지는가
Bind mount	host filesystem을 container namespace에 연결	host HTML v1/v2 응답 변경	host path 의존을 배포 환경에 가져가도 되는가
Named volume	Docker-managed persistent storage	paperclip-day2-data	container lifecycle과 data lifecycle을 분리했는가

End-Of-Day Checklist

- [] Dockerfile instruction 역할을 설명했다.
- [] docker build -t paperclip-static-site:day2 . 성공 또는 실패 evidence를 기록했다.
- [] docker run -p 18082:80으로 container를 실행했다.
- [] curl -I로 HTTP/1.1 200 OK를 확인했다.
- [] docker logs와 docker exec로 실행 증거를 확인했다.
- [] bind mount로 host 파일 수정이 container 응답에 반영되는 것을 확인했다.
- [] named volume이 container 삭제 후에도 데이터를 보존하는 것을 확인했다.
- [] container를 stop/rm으로 정리했다.
- [] README에 build/run/check/disk/cleanup/troubleshoot를 작성했다.

Next Connection

Day 3는 port, network, environment variable, volume을 다룬다. Day 2의 Dockerfile은 image를 만드는 기준이고, Day 3의 실행 옵션은 같은 image를 어떤 network/storage/config 조건으로 실행할지 결정하는 기준이다.